

Node.js Interview Q&A — Easy Spoken English

> Simple wording, first-person style, easy to say out loud in an interview.

Basics

****1. How does Node.js work?****

Node.js runs on one single thread, but it's event-driven. It uses the V8 engine, which is made by Google, to convert JavaScript into fast machine code. There's an event loop that handles things like file reading, timers, and requests, without blocking the main thread. Behind the scenes, there's a library called libuv, which manages a thread pool for background work like file access or network calls. Because of this non-blocking design, Node.js can handle thousands of requests at the same time without creating a new thread for each one.

****2. Can you access DOM in Node.js?****

No. DOM is a browser concept, used to interact with HTML pages. Node.js runs on the server side, so it never runs inside a browser, which means it has no access to DOM.

****3. What is V8 engine in Node.js?****

V8 is the JavaScript engine made by Google, written in C++. It's the same engine used inside Chrome. It converts JavaScript into machine code so it runs fast, and it also manages memory and garbage collection. Node.js uses V8 as its core, and adds extra APIs on top of it.

****4. Explain the engine Google uses for Node.js****

Google uses the V8 engine. It's written in C++, so it's very fast, and it lets JavaScript run outside the browser too. It's independent of any browser, has a huge community behind it, and works on many platforms.

****5. What do you mean by event loop in Node.js?****

The event loop is what lets Node.js handle many async tasks at once, even though it's single-threaded. It keeps listening for events, and whenever one happens, it runs the matching callback function.

****6. Why is Node.js single-threaded?****

Because JavaScript itself is asynchronous and non-blocking by nature. Keeping it single-

threaded makes development simpler, and it's still able to handle a lot of requests efficiently.

****7. If Node.js is single-threaded, how does it handle concurrency?****

It uses one thread with an event loop, but I/O operations run in the background asynchronously, so the main thread isn't blocked. Once a background task finishes, its callback gets added to a queue, and the event loop picks it up and runs it.

****8. What is control flow in Node.js?****

Control flow is basically the order in which async operations — like reading a file, calling an API, or querying a database — actually run and get their results handled. Since Node.js doesn't block and is event-driven, tasks don't always finish in the same order they started, so control flow makes sure everything is managed properly.

****9. What is the order in which control flow statements get executed?****

First, the task gets executed and put in a queue. Then the data is collected and stored. After that, concurrency is handled. And finally, the next lines of code run.

Why Node.js — Strengths, Weaknesses, Comparisons

****10. Why is Node.js preferred over Java or PHP?****

A few reasons — it's fast, especially for I/O-heavy work. It has NPM, which gives access to over 50,000 packages, so development is quicker. It's great for real-time apps since it doesn't wait around for APIs. You can use the same JavaScript code on both frontend and backend. And since it's JavaScript, web developers can pick it up easily.

****11. What are the pros and cons of Node.js?****

Pros: it handles many requests at once without blocking, thanks to non-blocking I/O. It's fast because of the V8 engine. And it has a big, active community.

Cons: being single-threaded, it's not great for heavy computation tasks. Async code can lead to "callback hell" if not managed well. And debugging async code can be tricky because stack traces are harder to follow.

****12. What are the main disadvantages of Node.js?****

It's single-threaded, so it doesn't fully use multi-core CPUs. It usually pairs with NoSQL databases rather than relational ones like MySQL. And since it updates often, sometimes those updates bring instability.

****13. Difference between Node.js and Angular?****

Node.js is a server-side runtime — it runs JavaScript outside the browser, mainly for backend work like APIs and databases. Angular is a frontend framework that runs in the browser and is used to build interactive UIs. Node.js uses JavaScript; Angular mostly uses TypeScript.

****14. Difference between Node.js and server-side languages like Python?****

Node.js is a runtime for JavaScript, and it's asynchronous by nature. Python is a full language that can be used server-side too, but it's synchronous by default, though it can do async with things like asyncio. Node.js is great for I/O-heavy, high-concurrency apps, while Python fits CPU-heavy work better.

Sync vs Async & Callbacks

****15. Difference between synchronous and asynchronous functions?****

Synchronous code blocks — one task must finish before the next one starts, and you get the result right away. Asynchronous code doesn't block — it starts a task and moves on, then later gives you the result through a promise, callback, or event. Sync is easier to handle with try-catch; async needs promises or async/await.

****16. How many types of API functions are there in Node.js?****

Two types — asynchronous non-blocking ones, mostly I/O operations that run outside the main loop, and synchronous blocking ones, which run directly in the main loop.

****17. Explain asynchronous and non-blocking APIs in Node.js.****

Asynchronous APIs let Node.js start something, like reading a file, and move on to the next task without waiting. Once it's done, a callback runs to handle the result. Non-blocking means the API doesn't stop the rest of the code from running while it waits — the event loop keeps things moving in the background.

****18. What is a callback function in Node.js?****

It's a function that runs after some task finishes. It stops things from blocking, so other code can keep running while we wait.

****19. What is callback hell?****

It happens when you nest callbacks inside callbacks inside callbacks. The code ends up

looking like a pyramid and becomes really hard to read. We usually fix this using promises.

****20. What are promises in Node.js?****

A promise is an object that represents whether an async task eventually succeeds or fails. It helps manage async code better than plain callbacks, and lets us chain things nicely using `.then()` and `.catch()`, which avoids callback hell.

****21. What are the three methods to avoid callback hell?****

Using `async/await`, using promises, or using generators.

The Module System

****22. What are modules in Node.js?****

A module is just a reusable piece of code that does one specific job, and it can live in one file or be spread across a few. It helps break the app into smaller, manageable pieces. Examples are `'http'`, `'fs'`, `'os'`, and `'path'`.

****23. What's the purpose of `'require'` in Node.js?**

`'require'` is used to bring in modules, whether built-in or external, into your app. For example, `const http = require('http')` imports the HTTP module.

****24. How do you import a module in Node.js?**

You can do it the CommonJS way, using `'require'`, like `const fs = require('fs')`. Or the modern ES module way, using `'import'`, like `import fs from 'fs'`.

****25. Which command is used to import external libraries?**

We use `'require()'` to bring in packages that were installed through npm, like `const express = require('express')`.

NPM & Package Management

****26. What is NPM?**

NPM stands for Node Package Manager. It's used to install and manage libraries and dependencies for a project. It relies on `'package.json'` and works through commands like `'npm install'`, `'npm update'`, and `'npm uninstall'`.

****27. What is package.json?****

It's a file that stores metadata about the project — things like its name, version, dependencies, scripts, and author info. It's basically the project's identity card.

****28. How do you install, update, and delete a dependency?****

You install with ``npm install package-name``, update with ``npm update package-name``, and remove with ``npm uninstall package-name``.

****29. How do you manage packages in your Node.js project?****

Mainly through npm — it lets me install third-party packages, and also create and publish my own packages if needed.

Building Your First Server

****30. How do you create a simple HTTP server in Node.js?****

I use the built-in ``http`` module. I create a server with ``http.createServer()``, write a response inside the callback, and then call ``server.listen()`` on a port, like 3000, to start it.

****31. How do you create a simple Express app in Node.js?****

First, make sure Node.js is installed. Then create a project folder, run ``npm init -y`` to set up ``package.json``, and install Express with ``npm install express``. Then create an ``app.js`` file, set up a basic route with ``app.get()``, and start the server with ``app.listen()``.

****32. What are the most commonly used libraries in Node.js?****

Express — a lightweight framework for building APIs, handling routes and middleware easily. And Mongoose — an ODM for MongoDB that helps manage schemas, relationships, and validation.

Environment Setup & Dev Tools

****33. How do you handle environment variables in Node.js?****

I use ``process.env`` to access them, and store the actual values in a ``.env`` file. To read that file, I install the ``dotenv`` package and call ``require('dotenv').config()`` at the top of my app.

****34. What's the purpose of NODE_ENV?****

It tells the app which environment it's running in — development, testing, or production —

so I can change behavior depending on the stage.

****35. What is REPL in Node.js?****

REPL stands for Read, Evaluate, Print, Loop. It's like an interactive shell where I can type JavaScript, it reads it, runs it, prints the result, and then waits for more input — good for quick testing and debugging.

****36. What tool is used for writing consistent code?****

ESLint. It helps keep the code style consistent, and it's actually built using Node.js itself.

Intermediate — Event-Driven Programming & EventEmitter

****37. What is event-driven programming in Node.js?****

It means the program reacts to events. There's a callback, or event handler, that runs when something happens, and there's an event loop that keeps listening for those events and triggers the right handler.

****38. What is an EventEmitter in Node.js?****

It's a class from the ``events`` module that lets objects emit events, and other parts of the code can listen for those events and react. It's basically Node's version of the observer pattern.

Timers & Event Loop Internals

****39. Explain the timers module in Node.js.****

It's global, so I don't need to import it. ``setTimeout()`` runs a function once after a delay. ``setImmediate()`` runs a function right after the current event loop cycle finishes, once I/O is done. ``setInterval()`` runs a function repeatedly with a fixed delay between each run.

****40. Difference between `setImmediate()` and `process.nextTick()`?**

``setImmediate()`` runs in the check phase, after I/O events are handled.

``process.nextTick()`` runs right after the current operation, before I/O, and it actually has higher priority. If overused, ``nextTick()`` can even block I/O.

****41. Difference between `setImmediate()` and `setTimeout()`?**

``setImmediate()`` runs right after the current phase of the event loop, with no real delay.

``setTimeout()`` waits for the specific delay I give it before running, even if I set it to 0ms.

HTTP, REST & Middleware

****42. What are the different types of HTTP requests?****

GET to fetch data, POST to create something new, PUT to fully update a resource, PATCH to partially update it, and DELETE to remove it.

****43. What is a REST API?****

REST stands for Representational State Transfer. It's a way for systems to talk to each other over the internet, usually exchanging JSON. It uses HTTP methods like GET, POST, PUT, DELETE to match CRUD operations on resources.

****44. What's the use of the url module in Node.js?****

It helps break a URL into readable parts, so different parts of the app can use them. The ``parse()`` method does that job.

****45. Explain middleware in Node.js.****

Middleware are functions that run during the request-response cycle. They can modify the request or response, handle errors, or even end the cycle early before it reaches the final route handler.

****46. What is body-parser in Node.js?****

It's middleware that parses the incoming request body before I handle it in my route — basically an npm package that processes data sent in HTTP requests.

****47. What is CORS in Node.js?****

CORS means Cross-Origin Resource Sharing. It's a browser-based mechanism that controls whether a server allows requests from a different origin. In Node.js, we usually use the ``cors`` npm package to handle this.

Buffers, Streams & Crypto

****48. What is a buffer in Node.js?****

Buffer is used to work with raw binary data. It's like an array, but it can't be resized, and it only deals with binary — each unit in it represents a byte. I can print a buffer with ``console.log()``.

****49. What are streams in Node.js?****

Streams let me handle data in small chunks instead of loading everything into memory at once — great for big files. There are four kinds: Readable, for reading data; Writable, for writing data; Duplex, which can do both; and Transform, which changes the data while reading and writing it, like compression.

****50. Explain the crypto module in Node.js.****

It's used for encrypting, decrypting, or hashing data, which adds a layer of security. Basically, it turns plain text into an encrypted format, and can decrypt it back when needed.

Child Processes

****51. What are child processes in Node.js?****

Since Node.js is single-threaded, one thread alone can't handle a heavy workload well. So the `child_process` module lets me spawn separate child processes, and they communicate with the main process through a built-in messaging system.

****52. What is fork in Node.js?****

`fork()` is a method to create a new child process, specifically for running another Node.js file. It helps share the workload by creating a new instance of the engine.

****53. Difference between spawn() and fork()?****

`spawn()` runs any system command or external program, and doesn't set up a communication channel by default. `fork()` is specifically for creating new Node.js processes, and it comes with a built-in messaging channel, so it's better for Node-to-Node communication.

Working with Databases

****54. How do you handle a database connection in Node.js?****

I use drivers or libraries depending on the database — for example, the MySQL driver for relational databases, or Mongoose for MongoDB. These give me methods to connect and run queries.

****55. How do you connect Node.js to MongoDB?****

After installing Mongoose, I call `mongoose.connect()` and pass in the database URL

along with some connection options.

****56. Explain the Redis module in Node.js.****

Redis is an open-source data store, often used as a cache, database, or message broker. It can hold data types like strings, hashes, sets, and more. It's popular with Node.js because it speeds things up and reduces load, and it's easy to integrate.

Testing & Security

****57. What is a test pyramid in Node.js?****

It's a way to structure tests. At the base, there are Unit tests — fast tests for individual functions. In the middle, Integration tests — checking how components work together, like an API talking to a database. At the top, End-to-End tests — testing the whole app flow, like simulating a user login, though these are slower and fewer.

****58. How do you validate data in Node.js?****

I usually use the `express-validator` package. There are other options like `joi`, but `express-validator` is the most common.

****59. Explain the passport module in Node.js.****

Passport is used to add authentication to an app — it helps handle sign-in and supports different strategies like Google or GitHub login.

****60. Explain the tls module in Node.js.****

It implements TLS and SSL protocols, built on OpenSSL, and it's used to set up a secure connection over the network.

Advanced — Sessions, Auth & File Uploads

****61. How do you manage sessions in Node.js?****

I use the `express-session` package. It stores session data as key-value pairs, and only the session ID is kept in the cookie, not the actual data.

****62. How do you implement authentication and authorization in Node.js?****

Authentication checks who the user is, and authorization checks what they're allowed to do. I usually use Passport for login strategies like OAuth or Google/GitHub, and JWT for token-based auth and role-based access.